
1. PRODUCT OVERVIEW

What is SnapFlip?

SnapFlip is a web-based SaaS tool for solo wedding photographers in India. A photographer logs in, uploads their wedding photos, picks background music, and instantly gets a cinematic page-turning digital flipbook — just like a real physical album but shareable via a link or QR code.

The client scans the QR or opens the link → sees a beautiful music-backed flipbook of their wedding → shares it with family on WhatsApp.

The Problem It Solves

Right now, photographers send photos via:

- Google Drive (ugly, no experience)
- WhatsApp (compressed, no order)
- Pen drive (physical, slow)

None of these feel premium. SnapFlip makes the delivery experience feel like a product — not a file transfer.

Who Uses It

Primary User: Solo wedding photographer in India

Secondary User: Their client (bride/groom) who views the flipbook

Market: 500,000+ wedding photographers in India

2. COMPLETE FEATURE LIST

2.1 LANDING PAGE (Public)

- Hero section: Live flipbook demo playing automatically
- Tagline: "Your wedding photos deserve better than a Drive link."
- CTA: "Create Your First Flipbook Free"
- How it works: 3 steps (Upload → Customize → Share)
- Pricing section
- Testimonials (after launch)
- Footer: Links, social, contact

2.2 AUTH SYSTEM (Day 7)

- Signup: Name, Email, Password, Studio Name (optional)
- Login: Email + Password
- Forgot Password: Email OTP reset
- Session: JWT token stored in localStorage
- Protected routes: Dashboard, Create Album, Album Detail

2.3 DASHBOARD (Photographer's Home)

- Header: Logo + Nav (Dashboard, Create Album, Pricing, Logout)
- Stats bar: Total Albums | Total Views | Plan (Free/Pro)
- Album Grid: Cards showing album cover, title, date, view count
- Each card: Open Flipbook | Copy Link | Copy QR | Delete
- Empty state: "No albums yet. Create your first one →"
- Create Album button (top right, always visible)

2.4 CREATE ALBUM FLOW (3 Steps)

STEP 1 — Upload Photos

- Drag & drop zone OR click to upload
- Accepts: JPG, PNG, WEBP
- Multiple file upload (up to 100 photos)
- Upload progress bar per photo
- Photo reorder via drag & drop after upload
- Delete individual photos
- Photos stored on Cloudinary

STEP 2 — Customize

- Album Title (text input, required)
- Client Name (optional)
- Wedding Date (date picker, optional)
- Music Picker (built-in, see Section 2.5)
- Cover photo selector (pick from uploaded photos)
- Flipbook theme: Classic / Dark / Minimal (v1: just Classic)

STEP 3 — Preview & Publish

- Live preview of flipbook before publishing
- Publish button → generates public URL + QR code
- Success screen: show link + QR + WhatsApp share button

2.5 MUSIC PICKER (Built-In, No External API)

- NO external API needed
- 10 curated royalty-free wedding background tracks bundled in the app
- Track categories: Romantic | Classical | Soft Bollywood | Instrumental

- Each track: Play preview (10 sec) | Select button
- Selected track plays automatically in the flipbook viewer
- Tracks stored in /public/music/ folder in the project

Track List (royalty-free, source from pixabay.com/music):

1. No Music (default)
2. Romantic Piano
3. Soft Strings
4. Classical Guitar
5. Bollywood Instrumental 1
6. Bollywood Instrumental 2
7. Wedding March (modern)
8. Acoustic Love
9. Cinematic Romance
10. Soft Flute

2.6 FLIPBOOK VIEWER (Core Feature)

- Built with StPageFlip.js (page-flip npm package)
- Realistic page-turn animation with shadow
- Auto-plays from first page OR shows page controls
- Controls: Previous | Next | Page number indicator
- Background music plays on first interaction (browser policy)
- Music control: play/pause button on viewer + volume slider
- Full screen mode button
- Mobile responsive: swipe to turn pages on mobile
- Share button: copy link + WhatsApp share
- Watermark on Free plan (bottom right: "SnapFlip.in")
- No watermark on paid plan

2.7 PUBLIC ALBUM PAGE (What Client Sees)

- No login required
- URL format: snapflip.in/view/[unique-slug]
- Shows: Album title, photographer name, flipbook viewer
- Music plays after first tap/click
- Share button (WhatsApp + copy link)
- "Create yours at SnapFlip.in" footer (subtle)

2.8 QR CODE

- Auto-generated after album is published
- Links to public album URL
- Download as PNG button (400x400px, white background)
- Photographer prints on album cover or visiting card

2.9 SUBSCRIPTION & PAYMENTS

Plans:

Plan	Price	Albums	Music	Watermark	QR Code
Free	Rs.0	1 album	No	Yes	Yes
Monthly	Rs.299/month	Unlimited	Yes	No	Yes
Yearly	Rs.2499/year	Unlimited	Yes	No	Yes

- Payment via Razorpay
- Subscription auto-renewal
- Cancel anytime
- Upgrade prompt when Free user tries to create 2nd album

3. SCREENS / PAGES LIST

#	Page	Route	Auth Required
1	Landing Page	/	No
2	Login	/login	No
3	Signup	/signup	No
4	Dashboard	/dashboard	Yes
5	Create Album	/create	Yes
6	Edit Album	/album/:id/edit	Yes
7	Album Detail	/album/:id	Yes
8	Public Flipbook	/view/:slug	No
9	Pricing	/pricing	No
10	Settings	/settings	Yes

4. TECH STACK (DETAILED)

Layer	Tech	Why
Frontend	React 18 + Vite	Fast build, hot reload
Styling	Tailwind CSS	Utility-first, fast to build
Routing	React Router DOM v6	SPA routing
Flipbook	page-flip (StPageFlip.js)	Best free flipbook lib for React
QR Code	qrcode.react	Simple, no backend needed
HTTP Client	Axios	API calls to backend

Backend	Node.js + Express	REST API	
Database	Supabase (PostgreSQL)	Free tier, auth built-in	
Storage	Cloudinary	Image hosting, CDN	
Music	Local files in /public	No API cost, royalty-free	
Payments	Razorpay	India-first, easy integration	
Hosting FE	Vercel	Free tier, auto-deploy from GitHub	
Hosting BE	Render	Free tier Node.js hosting	

5. DATABASE SCHEMA

Table: users

id UUID PK | name TEXT | email TEXT UNIQUE | password TEXT | studio_name TEXT | plan TEXT DEFAULT 'free' | plan_expiry TIMESTAMP | created_at TIMESTAMP

Table: albums

id UUID PK | user_id UUID FK | title TEXT | client_name TEXT | wedding_date DATE | slug TEXT UNIQUE | cover_url TEXT | music_track TEXT | is_published BOOLEAN | view_count INTEGER | created_at TIMESTAMP

Table: photos

id UUID PK | album_id UUID FK | url TEXT | order_index INTEGER | created_at TIMESTAMP

Table: subscriptions

id UUID PK | user_id UUID FK | razorpay_id TEXT | plan TEXT | status TEXT | started_at TIMESTAMP | expires_at TIMESTAMP

6. API ENDPOINTS

Auth:

POST /api/auth/signup

POST /api/auth/login

POST /api/auth/forgot

Albums (Auth Required):

GET /api/albums

POST /api/albums

GET /api/albums/:id

PUT /api/albums/:id

DELETE /api/albums/:id

POST /api/albums/:id/publish

Photos (Auth Required):

POST /api/albums/:id/photos

PUT /api/albums/:id/photos/order

DELETE /api/photos/:photoId

Public (No Auth):

GET /api/public/:slug

POST /api/public/:slug/view

Payments (Auth Required):

POST /api/payments/create-order

POST /api/payments/verify

POST /api/payments/webhook

7. UI DESIGN SYSTEM

Colors:

- Deep Teal #0B3037 — Primary background
- Teal Mid #134E5E — Cards, sections
- Navy Dark #141E30 — Secondary background
- Navy Mid #35577D — Hover states
- Glass White rgba(255,255,255,0.08) — Card backgrounds
- Glass Border rgba(255,255,255,0.15) — Card borders
- Text Primary #FFFFFF — Headlines
- Text Secondary #94A3B8 — Subtext, labels
- Accent #38BDF8 — CTA buttons, active states
- Success #22C55E — Published states
- Error #EF4444 — Error states

Typography:

- Display: Playfair Display 700 (48-64px)
- Heading: Inter 600 (24-32px)
- Body: Inter 400 (14-16px)
- Caption: Inter 400 (12px)
- Button: Inter 500 (14px)

Component Patterns:

- Cards: backdrop-blur-md, bg-white/5, border-white/10, rounded-2xl
- Buttons Primary: bg-sky-400, text-black, rounded-full, px-6 py-3

- Buttons Secondary: glass style, border-white/20
- Inputs: bg-white/5, border-white/10, rounded-lg, text-white

8. 7-DAY BUILD PLAN WITH ANTIGRAVITY PROMPTS

DAY 1 — Project Scaffold + Flipbook Viewer

Goal: Working flipbook in browser with 5 test images

ANTIGRAVITY PROMPT:

Create a React + Vite + Tailwind CSS project called snapflip at the current directory.

Install these npm packages: page-flip react-router-dom axios qrcode.react

Create this folder structure:

```
src/  
  components/  
    FlipbookViewer.jsx  
    Navbar.jsx  
  pages/  
    Home.jsx  
  assets/  
  styles/  
App.jsx  
main.jsx
```

Setup tailwind.config.js with custom colors:

- teal-deep: #0B3037
- teal-mid: #134E5E
- navy-dark: #141E30
- navy-mid: #35577D

In index.css add Google Font import for Inter and Playfair Display.

App.jsx: Single route "/" rendering Home.jsx. Background color #0B3037.

Home.jsx: Full screen dark page, centered heading "SnapFlip" in Playfair Display white, subtitle "Cinematic Wedding Albums" in Inter gray. Below that render FlipbookViewer component.

FlipbookViewer.jsx:

- Use HTMLFlipBook from page-flip
- Show 10 pages using these 5 Unsplash wedding images (2 pages each):
<https://images.unsplash.com/photo-1519741497674-611481863552?w=600>
<https://images.unsplash.com/photo-1606800052052-a08af7148866?w=600>
<https://images.unsplash.com/photo-1583939003579-730e3918a45a?w=600>
<https://images.unsplash.com/photo-1529636798458-92182e662485?w=600>
<https://images.unsplash.com/photo-1511285560929-80b456fea0bc?w=600>
- Each page: full image, object-cover, no border
- Page size: width 400, height 550
- Show Prev / Next buttons below flipbook
- Show page counter "Page X of Y"
- Container: glassmorphism card (bg white/5, backdrop-blur-md, rounded-2xl, border white/10)
- Buttons: rounded-full, bg-sky-400, text-black, px-5 py-2

Make sure it runs with `npm run dev` and shows the flipbook with working page turn animation.

DAY 2 — Photo Upload + Album Creation

Goal: Photographer can upload real photos to Cloudinary and save album

ANTIGRAVITY PROMPT:

Continue the snapflip project. Today we build photo upload + album creation. No auth yet — use a hardcoded temp user ID for now.

Backend setup first:

1. Create `/backend` folder with Node.js + Express
2. Install: `express cors dotenv multer cloudinary uuid`
3. Create `backend/.env`:
`PORT=5000`
`CLOUDINARY_CLOUD_NAME=lbchnngyv`
`CLOUDINARY_API_KEY=553368992287524`
`CLOUDINARY_API_SECRET=[from .env]`
4. Create `backend/config/cloudinary.js` — configure cloudinary with env vars
5. Create `backend/routes/upload.js`:
 - POST `/api/upload` — accepts multiple image files via `multer memoryStorage`
 - Uploads each to Cloudinary under folder "snapflip/albums"
 - Returns array of `{url, public_id}` for each uploaded image
6. Create `backend/server.js` — express app, cors, routes, listen on PORT

Frontend:

1. Create `src/pages/CreateAlbum.jsx` with 3-step wizard:

STEP 1 - Upload Photos:

- Large drag & drop zone (dashed border, icon, "Drop your wedding photos here")
- Click to browse files also works
- Accepts jpg/png/webp, multiple files
- After selecting: show photo grid thumbnails
- Each thumbnail has X button to remove
- Drag to reorder thumbnails
- Upload progress: show "Uploading X of Y..." with progress bar
- On upload: POST to /api/upload, get back Cloudinary URLs
- "Next →" button (disabled until at least 5 photos uploaded)

STEP 2 - Customize:

- Album Title input (required)
- Client Name input (optional)
- Wedding Date picker (optional)
- Music Picker component (see below)
- "← Back" and "Next →" buttons

STEP 3 - Preview & Publish:

- Show FlipbookViewer with the actual uploaded photos
- Show selected music name
- "← Back" and "Publish Album" buttons
- On Publish: save to localStorage for now (backend DB comes Day 7)
- Show success message with fake share link

MusicPicker component (src/components/MusicPicker.jsx):

- Show list of 10 music options as selectable cards
- Each card: music icon, track name, category tag, Play button
- Play button: plays 10-second preview from /public/music/ folder
- Selected card: highlighted with sky-400 border
- Default: "No Music" selected
- Tracks:

```
{id:'none', name:'No Music', category:'Default', file:null}  
{id:'romantic-piano', name:'Romantic Piano', category:'Romantic', file:'romantic-piano.mp3'}  
{id:'soft-strings', name:'Soft Strings', category:'Romantic', file:'soft-strings.mp3'}  
{id:'classical-guitar', name:'Classical Guitar', category:'Classical', file:'classical-guitar.mp3'}  
{id:'bollywood-1', name:'Bollywood Instrumental 1', category:'Bollywood',  
file:'bollywood-1.mp3'}  
{id:'bollywood-2', name:'Bollywood Instrumental 2', category:'Bollywood',  
file:'bollywood-2.mp3'}  
{id:'wedding-march', name:'Wedding March', category:'Classical', file:'wedding-march.mp3'}  
{id:'acoustic-love', name:'Acoustic Love', category:'Romantic', file:'acoustic-love.mp3'}  
{id:'cinematic', name:'Cinematic Romance', category:'Cinematic', file:'cinematic.mp3'}
```

```
{id:'soft-flute', name:'Soft Flute', category:'Instrumental', file:'soft-flute.mp3'}
```

NOTE: For now use placeholder silent mp3 files in /public/music/ — we will replace with real tracks separately.

Add route /create to App.jsx.

Add "Create Album" button on Home.jsx that navigates to /create.

Design: same dark teal glassmorphism style as Day 1. Step indicator at top showing Step 1 / 2 / 3.

DAY 3 — Music + Flipbook Integration

Goal: Real music plays inside the flipbook viewer

ANTIGRAVITY PROMPT:

Continue snapflip project. Today: music plays in the flipbook.

1. Update FlipbookViewer.jsx to accept props:

- images: array of URLs
- musicFile: filename string or null
- isPublic: boolean (shows watermark if true and no paid plan)

2. Inside FlipbookViewer:

- Create an audio element ref for the music
- Music auto-plays on first user interaction (click anywhere in flipbook)
- Show music control bar at bottom of flipbook:
 - * Play/Pause toggle button (icon)
 - * Track name
 - * Volume slider
- If musicFile is null: hide music controls
- Music loops continuously

3. Watermark:

- If isPublic=true: show "SnapFlip.in" text bottom-right, semi-transparent white, small font
- If isPublic=false: no watermark

4. Page turn sound:

- Play a soft page-turn sound on each page flip
- Use a very short mp3 in /public/sounds/page-turn.mp3

5. Update CreateAlbum Step 3 preview:

- Pass selected music to FlipbookViewer
- Music should play in preview

6. Update FlipbookViewer styling:

- Add soft drop shadow to the book
- Background behind book: dark gradient radial glow (teal glow)

Run and test that music plays when flipbook is interacted with.

DAY 4 — QR Code + Public Share Link

Goal: Each album gets a public URL and QR code

ANTIGRAVITY PROMPT:

Continue snapflip project. Today: public share link + QR code.

1. Create src/pages/PublicAlbum.jsx:

- Route: /view/:slug (no auth required)
- Load album from localStorage by slug
- Show: photographer name, album title, wedding date
- Render FlipbookViewer with actual photos and music
- isPublic=true (shows watermark)
- Below flipbook: WhatsApp share button + Copy Link button
- Footer: "Create your own at SnapFlip.in →"

2. Create src/components/QRCodeCard.jsx:

- Uses qrcode.react library
- Takes url prop
- Shows QR code 300x300, white background, rounded
- "Download QR" button: downloads as PNG using canvas
- "Copy Link" button: copies URL to clipboard, shows "Copied!" toast

3. After album publish (Step 3 of CreateAlbum):

- Generate a slug from album title + random 4 chars
- Save to localStorage: {slug, title, photos, music, createdAt}
- Success screen shows:
 - * "Album Published!" heading
 - * Public URL: snapflip.in/view/[slug]
 - * QR Code component
 - * Download QR button
 - * WhatsApp share button (opens wa.me with link + message)
 - * "Go to Dashboard" button

4. WhatsApp share message:

"Here's your wedding album

View it here: [URL]

Created with SnapFlip"

5. Add /view/:slug route to App.jsx

DAY 5 — Dashboard UI

Goal: Photographer's home to manage all albums

ANTIGRAVITY PROMPT:

Continue snapflip project. Today: full Dashboard UI.

1. Create src/pages/Dashboard.jsx at route /dashboard

2. Layout:

- Top: Navbar (Logo left, "Create Album" button right, Settings icon)
- Stats bar: 3 cards (Total Albums | Total Views | Current Plan)
- Albums grid: 3 columns desktop, 2 tablet, 1 mobile

3. Navbar (src/components/Navbar.jsx):

- Logo: "SnapFlip" in Playfair Display
- Links: Dashboard | Pricing
- Right: "Create Album" button (sky-400) + Avatar circle with initials

4. Album Card (src/components/AlbumCard.jsx):

- Cover image (first photo or placeholder)
- Album title, Client name, Wedding date
- View count (eye icon + number)
- Status badge: Published (green) / Draft (gray)
- Bottom actions: "Open" | "Copy Link" | "Copy QR" | "Delete"
- Hover: slight scale up, brighter border

5. Stats Cards: glassmorphism, icon + number + label

6. Empty state: SVG camera icon + "Create your first flipbook in 2 minutes" + CTA

7. Load albums from localStorage (temp until Day 7)

8. Delete: confirm dialog → remove from localStorage → remove from grid

9. "Upgrade to Pro" banner if on Free plan

DAY 6 — Razorpay Subscription

Goal: Working payment flow for Monthly/Yearly plans

ANTIGRAVITY PROMPT:

Continue snapflip project. Today: Razorpay subscription payments.

Backend additions (backend/routes/payments.js):

1. POST /api/payments/create-order

- Creates Razorpay order
- monthly: 29900 paise (Rs.299)
- yearly: 249900 paise (Rs.2499)
- Returns {orderId, amount, currency}

2. POST /api/payments/verify

- Verifies Razorpay signature
- Takes {razorpay_order_id, razorpay_payment_id, razorpay_signature, plan}
- On success: update user plan
- Returns {success: true, plan}

Add to backend/.env:

RAZORPAY_KEY_ID=your_test_key_id

RAZORPAY_KEY_SECRET=your_test_secret

Frontend:

1. Create src/pages/Pricing.jsx at route /pricing:

3 plan cards: Free | Monthly (Most Popular) | Yearly

2. On clicking Subscribe:

- Call POST /api/payments/create-order
- Open Razorpay checkout modal
(Key: RAZORPAY_KEY_ID, Name: SnapFlip, Theme: #38BDF8)
- On success: call POST /api/payments/verify
- Show "Plan Activated!" toast, navigate to dashboard

3. Add Razorpay script in index.html:

```
<script src="https://checkout.razorpay.com/v1/checkout.js"></script>
```

NOTE: Use Razorpay Test Mode keys for now.

DAY 7 — Supabase Backend + Auth + Deploy

Goal: Real database, real auth, live on Vercel

ANTIGRAVITY PROMPT:

Continue snapflip project. Today: real backend with Supabase + auth + deploy.

1. Create these tables in Supabase:

```
CREATE TABLE users (  
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  name TEXT NOT NULL,  
  email TEXT UNIQUE NOT NULL,  
  password TEXT NOT NULL,  
  studio_name TEXT,  
  plan TEXT DEFAULT 'free',  
  plan_expiry TIMESTAMP,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE TABLE albums (  
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
  title TEXT NOT NULL,  
  client_name TEXT,  
  wedding_date DATE,  
  slug TEXT UNIQUE NOT NULL,  
  cover_url TEXT,  
  music_track TEXT DEFAULT 'none',  
  is_published BOOLEAN DEFAULT false,  
  view_count INTEGER DEFAULT 0,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE TABLE photos (  
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  album_id UUID REFERENCES albums(id) ON DELETE CASCADE,  
  url TEXT NOT NULL,  
  order_index INTEGER NOT NULL,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE TABLE subscriptions (  
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  user_id UUID REFERENCES users(id),  
  razorpay_id TEXT,  
  plan TEXT NOT NULL,  
  status TEXT DEFAULT 'active',  
  started_at TIMESTAMP DEFAULT NOW(),  
  expires_at TIMESTAMP  
);
```

2. Backend:

Install: `@supabase/supabase-js bcryptjs jsonwebtoken`

Add to backend/.env:

```
SUPABASE_URL=your_url  
SUPABASE_ANON_KEY=your_key  
JWT_SECRET=snapflip_secret_2026
```

Create all API routes replacing localStorage with real Supabase queries.

Create auth middleware to verify JWT on protected routes.

3. Frontend:

Create Login.jsx and Signup.jsx pages

Create AuthContext.jsx to store JWT + user

Protected route wrapper (redirect to /login if no token)

Replace all localStorage calls with real Axios API calls

4. Deploy:

- Frontend: push to GitHub → connect to Vercel → set env vars → deploy

- Backend: deploy to Render.com → set env vars → deploy

- Update VITE_API_URL in Vercel to point to Render backend URL

Full test after deploy:

Signup → Create Album → Upload → Pick Music → Publish → QR → Public View → Subscribe
→ Upgrade

9. ENVIRONMENT VARIABLES

Frontend (.env):

```
VITE_CLOUDINARY_CLOUD_NAME=lbchngyv
```

VITE_CLOUDINARY_API_KEY=553368992287524
VITE_API_URL=http://localhost:5000
VITE_RAZORPAY_KEY_ID=your_test_key

Backend (backend/.env):

PORT=5000
NODE_ENV=development
CLOUDINARY_CLOUD_NAME=lbchngyv
CLOUDINARY_API_KEY=553368992287524
CLOUDINARY_API_SECRET=your_secret
SUPABASE_URL=your_supabase_url
SUPABASE_ANON_KEY=your_anon_key
JWT_SECRET=snapflip_secret_2026
RAZORPAY_KEY_ID=your_test_key_id
RAZORPAY_KEY_SECRET=your_test_secret

10. SECURITY CHECKLIST

- [] Supabase RLS enabled on all tables
- [] Cloudinary signed uploads only
- [] JWT tokens expire in 7 days
- [] Passwords hashed with bcrypt 10 rounds
- [] Razorpay webhook signature verified server-side
- [] API keys never in frontend code
- [] CORS restricted to snapflip.in in production
- [] Rate limit upload endpoint (max 100 photos per request)
- [] Input validation on all API endpoints
- [] .env files in .gitignore

11. LAUNCH PLAN

Day 8: Record 60-second demo Reel showing the flipbook
Day 9: Post Reel on Instagram + tag 10 photography accounts
Day 10: DM 50 wedding photographers — "Beta access free for 30 days"
Day 11: Post in Facebook groups — Wedding Photographers India
Day 12: Share in WhatsApp photography communities
Goal: 10 paying users in first 30 days
Beta Price: Rs.99/month for first 50 users
Upgrade Price: Rs.299/month after 50 users

SnapFlip PRD v1.0 — Built with Claude + Antigravity